



Generating Synthetic Handwriting with Diffusion Models for Word-Level Writer Identification and Retrieval

Master Thesis

Hamza Yazbeck

University of Bern

July 2026



Abstract

Writer identification and writer retrieval ask, respectively, who wrote a sample of handwriting and which other samples came from the same hand. Modern systems for both are trained with deep neural networks, and those networks need large amounts of labelled handwriting, which is time-consuming and requires domain experts. Therefore, the thesis investigates whether synthetic handwriting from a latent diffusion model can ease that dependence on real data.

We generate word images with DiffusionPen, a handwriting-text-generation model whose style encoder conditions synthesis on a target writer, and use those images to train GR-RNN, a word-level writer-identification network that we also adapt for retrieval. The study works at the word level on both contemporary (CVL) and historical (Bullinger) handwriting, and treats identification and retrieval as co-equal tasks. We look at to what extent synthetic data can be used to train these models, in particular whether it augments a real-only baseline, how the benefit scales with volume, how it depends on image representation and on the amount of data used to train the generator, and how it carries across data-availability regimes, domains, tasks, and retrieval granularities.

The central finding is that the value of synthetic handwriting is governed by image representation, and behind that by which aspects of a writer’s hand the generator reproduces. Synthetic augmentation consistently helps models that consume binarised images, with the largest relative gains when real data is scarce, but is neutral or slightly harmful for models that consume grayscale images. We put this down to the style encoder, which captures handwriting style (the shape and slant of strokes) rather than low-level ink and pen artefacts: the generated samples carry writer style rendered in generic ink, which helps binarised models that cannot use ink cues while taking away the cues grayscale models lean on. The same pattern holds on historical handwriting, and, for retrieval, document-level aggregation clearly outperforms word-level matching.

Quantitatively, synthetic augmentation raises binarised Top-1 identification accuracy on CVL from 82.87% to 84.07% at twice the real training volume, with its largest low-resource gain of about 7.5 points when only a tenth of the real data is available; document-level retrieval mAP rises for both representations, to 88.12% for binarised and 90.58% for grayscale. Grayscale identification, already near 94%, neither gains nor loses meaningfully, in keeping with the representation-dependent pattern above. A train-on-real, test-on-generated check bears this out directly: binarised generated words are attributed to the correct writer about 78% of the time, against about 9% for grayscale.

Supervisor: Andreas Fischer, AIBEX Group

Advisor: Marco Peer, AIBEX Group

Contents

Abstract	1
1 Introduction	4
1.1 Motivation	4
1.2 Problem Statement and Scope	5
1.3 Contributions	6
1.4 Research Questions	6
1.5 Outline	7
2 Related Work	8
2.1 Deep Writer Identification	8
2.2 Writer Retrieval	9
2.3 Handwriting Synthesis	9
2.4 Synthetic Data for Training	10
3 Methodology	11
3.1 Preliminaries	11
3.1.1 Image Representations of Handwriting	11
3.1.2 Diffusion Models	11
3.2 Pipeline Overview	12
3.3 Handwriting Generation with DiffusionPen	13
3.4 Synthetic Data Generation	15
3.5 Writer Identification Model: GR-RNN	15
3.6 Retrieval Adaptation	17
3.7 Datasets and Splits	18
4 Experimental Setup	19
4.1 Preprocessing	19
4.2 Training Configurations	19
4.2.1 Style Encoder	20
4.2.2 Diffusion Model	20
4.2.3 Writer Identification Model	20
4.3 Training Dynamics	20
4.4 Evaluation Metrics	22

4.5	Reproducibility	22
5	Writer Identification Results	23
5.1	Style Preservation	23
5.2	Effect of Image Representation	24
5.3	Effect of Synthetic-Data Volume	25
5.4	Effect of Real-Data Availability	26
5.5	Historical Handwriting: SBullinger	27
5.6	Robustness to Random Seed	27
5.7	Discussion	28
6	Writer Retrieval Results	30
6.1	Word-Level versus Document-Level Retrieval on CVL	30
6.2	Effect of Representation in Retrieval	32
6.3	Historical Handwriting: SBullinger	32
6.4	Discussion	33
7	Conclusion and Future Work	34
7.1	Summary of Findings	34
7.2	Future Work	35
8	Declaration on the Use of Generative AI	37
	References	38

Chapter 1

Introduction

1.1 Motivation

Handwriting is a behavioural biometric. The way a person forms letters, joins strokes, and spaces words is built up over years of motor learning and habit, and it stays individual enough to attribute a sample to its writer from very little material, sometimes a single word (He and Schomaker, 2020). This individuality underlies applications in forensic document examination, historical manuscript research, and the large-scale study of archival collections (Peer et al., 2024).

The thesis investigates two tasks built on this signal, both at the word level, taking a single handwritten word image as the unit of input. Writer Identification (WI) assigns a query sample to one of a known set of writers; because the writer is assumed to belong to that set, it is a closed-set classification problem. Writer Retrieval (WR) is its open-set analogue: given a query, it ranks a gallery of documents by how likely each is to share the query’s hand, and the author of the query need not appear in the gallery, nor among the writers seen during training. The word level is the most data-hungry of the granularities, because a single word carries less context than a line or a page, so more labelled samples per writer are needed to learn a representation that separates one hand from another. It is at the same time the most flexible, since word images can be cropped from almost any document without reliable line- or page-level layout.

Both tasks are dominated by deep learning. A modern identification network needs many labelled examples per writer to learn such a representation, yet in the situations where identification is most useful that data is scarce. A forensic case may turn on a handful of words from a questioned writer, and historical archives hold only short, degraded fragments from each scribe, written over decades during which that scribe’s own hand drifted; collecting and labelling more handwriting is slow and costly, and for writers long dead it is impossible (Peer et al., 2024, 2026).

Generative modelling offers a way to close this gap. Handwritten Text Generation (HTG) synthesises handwriting images from a target text and, optionally, a reference style. For much of its history the field relied on generative adversarial networks (GANs), which produce plausible images but are unstable to train, prone to the mode collapse that narrows their diversity, and only loosely faithful to a writer’s style (Alonso et al., 2019; Fogel et al., 2020; Kang et al., 2020). Diffusion models have improved on this: their samples are sharper, more varied, and truer to the target style, to the point that a writer classifier trained on real handwriting attributes diffusion-generated words to the correct writer far more reliably than it does GAN-generated

ones (Nikolaidou et al., 2023). If generated samples preserve a writer’s style, they become a source of training data that can be produced in any quantity, for any writer, with any text. The thesis examines whether, and under what conditions, this holds for word-level identification and retrieval.

1.2 Problem Statement and Scope

The thesis asks whether, and under what conditions, diffusion-generated word images can serve as training data for word-level writer identification and retrieval. The study is restricted to offline, word-level handwriting, and does not address online (trajectory-based) input or line- and page-level recognition. The generator throughout is DiffusionPen (Nikolaidou et al., 2024), a latent diffusion model built for few-shot styled handwriting generation; the downstream model for both tasks is the Global-Context Residual Recurrent Neural Network (GR-RNN) of He and Schomaker (2021), a strong word-level WI architecture. Experiments run on two datasets at opposite ends of the difficulty range: the contemporary CVL database (Kleber et al., 2013) and a subset of the historical BullingerDB collection (Peer et al., 2026). DiffusionPen and GR-RNN are treated as fixed building blocks; we do not propose new generative or recognition architectures, and the contribution lies in understanding how the data produced by the former can be used to train the latter.

We use synthetic data in two roles. In the first, the model is trained on generated data alone, which measures how much of the writer signal synthetic handwriting carries by itself. In the second, augmentation, the generated data is added to the real training set, which tests whether it improves on a real-only baseline. Both roles are evaluated on the two tasks side by side: for identification, Top-1 and Top-5 accuracy over a fixed set of writers; for retrieval, mean average precision (mAP) and Top- k over query and gallery writers that do not overlap. Retrieval is evaluated at two granularities, word-level matching, where every word image is a query against a gallery of single words, and document-level matching, where the per-word embeddings of a document are pooled into a single writer signature.

A sample of handwriting carries two kinds of information: writing style and low-level rendering artefacts. The writing style is the geometry of how strokes and letters are formed, their slant, curvature, proportions, and relative placement, and it is the writer-specific signal that handwriting biometrics target. The rendering artefacts are incidental properties such as ink intensity, contrast, and whatever the particular pen and paper contributed; these can correlate with writer identity within a single dataset without being intrinsic to the handwriting. DiffusionPen conditions generation on a style embedding produced by a dedicated style encoder, a MobileNetV2 trained with a combined triplet and classification objective (Sandler et al., 2018; Nikolaidou et al., 2024). Because the encoder is trained by metric learning, and has been shown to carry its style representation across scripts, it captures stroke-level style rather than the ink-and-pen artefacts of any single sample.

We therefore expect synthetic images to reproduce a writer’s style while not reproducing the original ink and pen cues, and the experiments confirm this: synthetic augmentation helps models trained on binarised images, where ink intensity has been discarded and only stroke shape remains, and does little or nothing for models trained on grayscale images, which can still

exploit the ink-and-pen cues that synthetic data omits. Concretely, the thesis quantifies how this interaction between image representation and the usefulness of synthetic data behaves as the amount of synthetic data grows, as real data becomes scarce, across the contemporary and historical datasets, and across identification and retrieval. Although DiffusionPen can interpolate between styles to synthesise entirely new writers, we generate extra samples only for writers who already exist, and leave the generation of novel synthetic writers to future work.

1.3 Contributions

This thesis makes three main contributions.

1. A study of diffusion-based synthetic augmentation that spans both writer identification and writer retrieval at the word level. As far as we are aware, this is the first work to evaluate diffusion-generated word images as training data across both the closed-set (identification) and open-set (retrieval) paradigms, and the first to apply diffusion-based handwriting synthesis to writer retrieval on historical documents.
2. A characterisation of the conditions under which synthetic data is most beneficial. We measure the effect of synthetic-data volume and show that the relative benefit of augmentation grows as the amount of real data shrinks, which is exactly the low-resource regime that motivates the work. We also show that the usefulness of the generated data depends strongly on how much real data was used to train the generator.
3. An evaluation across contemporary and historical handwriting, with a reproducible protocol. We report results on the contemporary CVL database and the historical BullingerDB subset, including word- and document-level retrieval, and we release our train/test splits and finetuned DiffusionPen checkpoints to support reproduction and further work.

1.4 Research Questions

The work is organised around three research questions, each with two sub-questions that line up with the experiments in Chapters 5 and 6.

RQ1 (Benefit). To what extent, and in what way, do diffusion-generated synthetic word images improve word-level writer identification and retrieval when added to real training data?

- **RQ1.1 (Augmentation).** How does adding synthetic word images to the real training set affect identification and retrieval relative to a real-only baseline?
- **RQ1.2 (Volume).** How does the benefit change as the amount of synthetic data added grows?

RQ2 (Data dependence). How does the usefulness of synthetic data depend on how much real data is available, both to the downstream model and to the generator?

- **RQ2.1 (Low-resource).** How does the relative benefit of augmentation change as real training data becomes scarce?
- **RQ2.2 (Generator data).** How does the amount of real data used to train the generator affect the usefulness of its synthetic samples?

RQ3 (Generality). How far does the usefulness of synthetic data carry across domains, tasks, and retrieval granularities?

- **RQ3.1 (Domain).** To what extent does the benefit transfer from contemporary (CVL) to historical (SBullinger) handwriting, and how do the two regimes differ?
- **RQ3.2 (Task and granularity).** How does the benefit differ between identification and retrieval, and, within retrieval, between word-level and document-level evaluation?

One further factor is reported alongside these questions, though it is not posed as a question in its own right: image representation, both the format in which handwriting is synthesised (RGB, grayscale, binarised) and the format the model consumes (grayscale, binarised). Chapters 5 and 6 examine it directly, and the discussion relates it to handwriting style: the generator reproduces a writer’s style but not the incidental ink and pen, which makes synthetic data valuable to models that must rely on style and largely inert to those that can lean on ink instead.

1.5 Outline

The rest of the thesis is organised as follows. Chapter 2 reviews related work on deep writer identification and retrieval, on handwriting synthesis, and on the use of synthetic data for training. Chapter 3 sets out the methodology: the image representations and diffusion-model background the approach builds on, the DiffusionPen generation pipeline, the procedure for synthesising training data, the GR-RNN model and the change that adapts it for retrieval, and the datasets and task-specific splits. Chapter 4 covers the experimental setup, from preprocessing through training configurations to evaluation metrics. Chapters 5 and 6 present and discuss the results, for identification and retrieval respectively, on both datasets. Chapter 7 draws the conclusions together and sets out directions for future work.

Chapter 2

Related Work

This chapter reviews the work most relevant to the thesis: deep approaches to writer identification and retrieval, methods for synthesising handwriting, and earlier uses of synthetic data to train recognition models.

2.1 Deep Writer Identification

The first deep-learning approach to writer identification came from Fiel and Sablatnig (2015), who trained a convolutional network on handwriting and took the activations of its penultimate layer as a feature vector, then pooled those features over a document for identification and retrieval. The method already reached high identification accuracy on contemporary benchmarks such as CVL.

The literature can be divided into two directions. The first is the level at which the image is presented to the network: a whole page or text line carries more context but produces variable-sized, unevenly filled inputs, whereas a single word or fragment yields many more training samples per document, at the cost of less context per sample. The second is how the per-sample features are turned into one descriptor for a document, whether by simple averaging, by more elaborate encodings that pool many local descriptors into a fixed-length vector, or by a recurrent module that reads a sequence of parts. The methods discussed below differ mainly in how they resolve these two choices, and the word-level, sequence-aware design of GR-RNN adopted in this thesis is one particular point in that space.

He and Schomaker (2020) introduced FragNet, which rests on the observation that even small local fragments of a word carry enough information to tell writers apart. FragNet cuts overlapping patches from a word image, runs each through a shared CNN, and pools the patch embeddings into a fixed-length representation. Working on fragments deals with variable-length input naturally, doubles as a form of data augmentation, and holds up well against localised degradation.

Their follow-up, and the model this thesis builds on, is the Global-Context Residual Recurrent Neural Network (GR-RNN) (He and Schomaker, 2021). GR-RNN adds a sequential component to the fragment-based idea: a writer’s style shows up in individual strokes but also in regularities that run across a sequence of fragments. The model feeds the fragments of a word through a gated recurrent unit and joins the recurrent output to a global-context vector, the global average pool

of the final convolutional feature map, through a residual connection, so that each fragment’s representation carries both local detail and the global picture. GR-RNN reports leading word-level accuracy on contemporary datasets such as CVL, which is why we adopt it here. Chapter 3 describes the architecture in detail.

More recently, Koepf et al. (2022) looked at vision transformers for identification and retrieval on forensic documents at the page level, and found their usefulness held back by the small size of typical handwriting datasets, an observation that sharpens the case for data augmentation made in this thesis. The ICDAR 2026 CircleID competition (Gorges et al., 2026) studies identification and pen classification from hand-drawn circles, including spotting unseen (outlier) writers.

2.2 Writer Retrieval

Writer retrieval is the open-set counterpart to identification, and work on it increasingly targets historical collections, where the writer population is large and open-ended and where degradation and stylistic drift make the problem harder. A representative example is SAGHOG (Peer et al., 2024), a self-supervised method that pre-trains an autoencoder to reconstruct HOG (histogram of oriented gradients) features and then fine-tunes the representation for retrieval, with strong results on historical material. Pipelines of this kind generally learn a feature extractor and aggregate per-fragment or per-patch descriptors into a document-level signature, often through an encoding such as VLAD, and then rank by nearest neighbour over those signatures. The historical BullingerDB dataset (Peer et al., 2026) used in this thesis was released alongside handwriting-recognition and retrieval benchmarks, and provides a large-scale open-set retrieval setting.

Two themes run through retrieval work and bear directly on this thesis. The first is aggregation: because a single word or fragment is a weak and noisy cue to identity, retrieval systems combine many local descriptors into a document-level signature before ranking, and the quality of that aggregation often matters as much as the underlying features (Fiel and Sablatnig, 2015; Peer et al., 2024). The second is supervision: labelled historical data is scarce, so recent methods lean on self-supervised pre-training to learn a representation from unlabelled script before any writer labels are used (Peer et al., 2024). Both themes recur in our own results, where document-level aggregation is what turns weak word-level retrieval into strong document-level retrieval, and where the scarcity of labelled historical data is the problem that synthetic data is meant to ease.

2.3 Handwriting Synthesis

GANs dominated offline handwriting generation for a long time. The earliest methods condition mainly on text: Alonso et al. (2019) encode the target string with a recurrent network and lean on an auxiliary CTC recogniser to keep the output legible, while ScrabbleGAN (Fogel et al., 2020) pairs character-level filters with a text recogniser to vary style and stroke width. Control over writer style, in both, is limited. GANwriting (Kang et al., 2020) brings in joint text-and-style conditioning, taking a handful of style reference images along with the text and training adversarially against both a recogniser and a writer classifier, which lets it generate in seen

and unseen styles; Davis et al. (2020) produce full lines of arbitrary length by predicting inter-character spacing from a style reference. A separate strand marries transformer encoder-decoder architectures with few-shot style encoding, VATr (Pippi et al., 2023) among them, which reads style from reference samples and generates words through cross-attention. The same problems recur across all of these: unstable adversarial training, thin diversity from mode collapse, and style that is preserved only loosely.

Moving to diffusion models brought a clear jump in both quality and style fidelity. WordStylist (Nikolaidou et al., 2023) is a latent-diffusion model for styled word generation that used writer identification itself as an evaluation: a writer classifier attributed its generated samples far more accurately than it did GAN-generated ones, which shows how much better diffusion preserves writer characteristics. Its successor, DiffusionPen (Nikolaidou et al., 2024), is the generator we use. DiffusionPen is a few-shot, style-conditioned latent-diffusion model, and its distinguishing feature is a dedicated style encoder trained with a hybrid metric-learning and classification objective, which yields a continuous style space that supports both faithful conditioning and interpolation between writers. Beyond the word level, Mayr et al. (2025) tackle zero-shot, paragraph-level imitation with latent diffusion, a sign that diffusion-based HTG is maturing across granularities. DiffusionPen is described in full in Chapter 3.

2.4 Synthetic Data for Training

Using generated handwriting as training data, rather than treating generation as the end in itself, has been studied most carefully for handwriting text recognition (HTR). Muth et al. (2024) show that synthetic text can sharply improve the data efficiency of HTR models, and, in particular, that the relative payoff of synthetic augmentation grows as real training data becomes scarce. That result is a direct spur to the low-resource study here, and a recognition-side analogue of the identification- and retrieval-side questions we ask. For writer identification specifically, generated samples as training data remain comparatively under-explored; to the best of our knowledge they have not been studied at the word level across both identification and retrieval, nor for historical writer retrieval, which is the gap this thesis sets out to fill.

More broadly, training models on generated or simulated data has a long track record in computer vision, where it is used whenever real labelled data is expensive, sensitive, or rare, and where the central question is how well a model trained on synthetic data transfers to real data. Handwriting is a favourable case for the idea, because a generator that captures a writer’s style can in principle manufacture an unlimited supply of labelled examples of that writer. The open question, and the one this thesis addresses, is whether that promise survives contact with the details: which image representation the downstream model consumes, how much real data the generator itself needs to be useful, and whether any benefit carries from clean contemporary handwriting to degraded historical material and from identification to retrieval.

Chapter 3

Methodology

This chapter describes the approach: the image representations and diffusion-model background it builds on, the two-stage pipeline, the DiffusionPen generator and the procedure for synthesising training data, the GR-RNN identification model and the change that adapts it for retrieval, and finally the datasets and task-specific splits.

3.1 Preliminaries

This section sets out two pieces of background the rest of the chapter relies on: the representations a handwriting image can take, and the diffusion-model framework underlying DiffusionPen.

3.1.1 Image Representations of Handwriting

A scanned word image can be presented to a model in three representations, and the choice fixes what information the model can use. A colour (RGB) image keeps the full appearance of the writing, including ink colour and the texture of the paper or parchment. A grayscale image discards colour but keeps intensity, so it still records ink darkness and stroke contrast. A binarised image collapses every pixel to ink or background, keeping only stroke shape and discarding intensity. We binarise with Sauvola’s adaptive thresholding (Sauvola and Pietikäinen, 2000), which sets a local threshold from the mean and standard deviation in a sliding window and copes well with the uneven illumination and degraded backgrounds common in historical material.

As set out in Section 1, binarisation is what removes the incidental ink-and-pen characteristics, leaving a model with writing style alone, whereas a grayscale model can also exploit ink-and-pen cues that correlate with a writer within a dataset. This is why the representation the downstream model takes is decisive for the value of synthetic data, a point Chapters 5 and 6 examine.

3.1.2 Diffusion Models

DiffusionPen belongs to the family of diffusion models, latent-variable generative models that learn to reverse a fixed forward process that gradually turns data into noise; generation then runs the learned reverse process from noise back to data. This section summarises the three developments the generator relies on.

Denoising diffusion probabilistic models (DDPMs; Ho et al., 2020) set up the forward process as a Markov chain that adds Gaussian noise at each of T steps. Starting from a data sample x_0 , one step is

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}), \quad (3.1)$$

where the variance schedule $\{\beta_t\}_{t=1}^T$ climbs from near zero to near one. Writing $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, the marginal at any step is available in closed form,

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (3.2)$$

so a noised sample at any timestep can be drawn directly from x_0 . Generation needs the reverse process $p_\theta(x_{t-1} | x_t)$, which is modelled as Gaussian and parameterised by a neural network; Ho et al. (2020) show that training reduces to teaching a network $\epsilon_\theta(x_t, t)$ to predict the added noise by minimising

$$\mathcal{L} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]. \quad (3.3)$$

Faithful sampling is slow, because the Markovian reverse process takes hundreds or thousands of network evaluations.

Denoising diffusion implicit models (DDIMs; Song et al., 2021) reduce that cost. The DDPM objective depends only on the marginals $q(x_t | x_0)$, not on the particular Markov chain, so DDIM defines a non-Markovian forward process with the same marginals and reuses a network trained as a DDPM unchanged. Its reverse process can be made deterministic and run over a subsequence of the original timesteps, so good samples come out in far fewer steps, tens rather than hundreds. DiffusionPen uses a DDIM scheduler with as few as 50 reverse steps at sampling time.

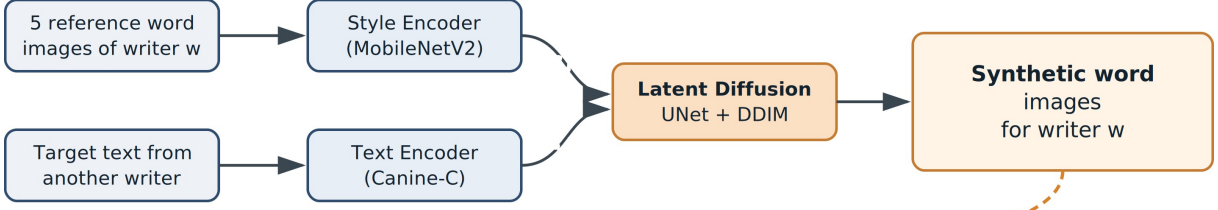
Latent diffusion models (LDMs; Rombach et al., 2022) cut the cost further by running the diffusion in the compressed latent space of a pre-trained autoencoder rather than in pixel space. An encoder $\mathcal{E}$ maps an image x to a latent $z = \mathcal{E}(x)$, the forward and reverse processes act on z , and a decoder $\mathcal{D}$ returns an image $\hat{x} = \mathcal{D}(z)$. Because the latent is spatially much smaller, both training and sampling are considerably cheaper while perceptual quality is largely preserved. LDMs also admit conditional generation by feeding external conditions, such as a text prompt or a style embedding, into the denoising network through cross-attention. DiffusionPen is exactly this kind of conditional latent diffusion model, conditioned jointly on text and writer style.

3.2 Pipeline Overview

The approach runs in two stages, sketched in Figure 3.1. In the generation stage, DiffusionPen is trained on a dataset’s real training data and then used to synthesise extra word images, each conditioned on a target writer’s style and a target text. In the training stage, those synthesised images train the GR-RNN writer model, either on their own (the replacement setting) or alongside the real training data (the augmentation setting); the trained model is then evaluated on a fixed real test set, as a closed-set classifier for identification and as an embedding extractor for open-set retrieval. Every result is measured against a real-data-only baseline.

Three training conditions recur throughout the experiments and are worth naming once here.

Stage 1: Handwriting generation (DiffusionPen)



Stage 2: Writer-model training and evaluation



Figure 3.1: Overview of the two-stage pipeline. In Stage 1, DiffusionPen generates word images for a writer, conditioned on five style reference images and a target text drawn from another writer. In Stage 2, the generated images train the GR-RNN model, which is evaluated for closed-set identification and open-set retrieval.

The real-only baseline trains GR-RNN on the real training data alone, with no synthetic images, and is the reference against which every other condition is compared. The synthetic-only, or replacement, condition trains on generated images alone, with no real data, and measures how far synthetic data can stand in for real data. The augmentation condition trains on the real data together with a quantity of generated data, and measures whether synthetic images add anything on top of the real ones. Reporting the three side by side keeps two distinct questions apart: whether synthetic data can replace real data, and whether it can usefully augment it.

3.3 Handwriting Generation with DiffusionPen

The generator is DiffusionPen (Nikolaidou et al., 2024). It is a conditional latent diffusion model for few-shot, style-conditioned word generation, with three parts (a latent autoencoder, a text encoder, and a style encoder) whose outputs condition a denoising UNet. Figure 3.2 shows the structure.

The latent autoencoder is a pre-trained variational autoencoder that maps a word image to a compact latent, where the diffusion runs, and decodes generated latents back to pixel-space word images. Running the diffusion in this latent rather than in pixel space is what keeps training and sampling tractable (Section 3.1.2).

The text encoder is Canine-C (Clark et al., 2022), a character-level, tokenisation-free language encoder that works directly on raw character sequences and encodes the target word. We use it as released in the original DiffusionPen implementation (github.com/koninik/DiffusionPen) and keep it fixed, without training it ourselves. Having no fixed vocabulary, it can render out-of-vocabulary words without losing legibility, which matters when the target transcriptions are arbitrary. The encoded sequence becomes the text condition c_T .

The style condition c_s comes from a style encoder SE , a MobileNetV2 backbone (Sand-

Training

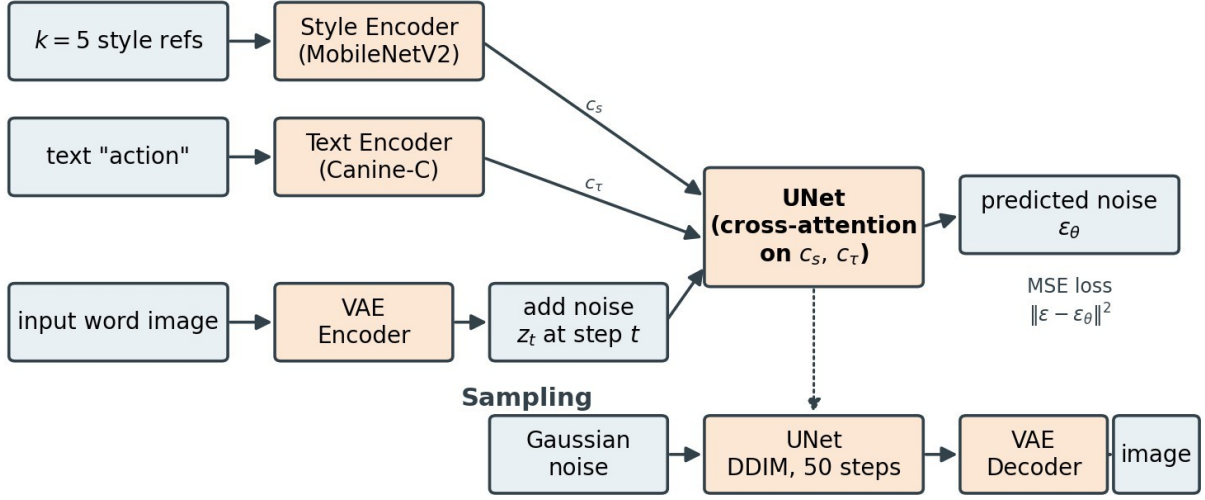


Figure 3.2: DiffusionPen architecture. A style encoder and a text encoder produce conditions c_s and c_τ ; during training a VAE encodes the image to a latent that is noised and denoised by a conditioned UNet; during sampling, a DDIM scheduler denoises Gaussian noise in 50 steps and the VAE decoder produces the final word image.

ler et al., 2018) trained with a hybrid objective: a classification loss together with a metric-learning triplet loss. Given an anchor image of a writer, a positive sample from the same writer, and a negative from a different one, the triplet loss pulls same-writer embeddings together and pushes different-writer embeddings apart, while the classification loss keeps the embedding writer-discriminative. What comes out is a continuous, well-structured style space in which closeness means stylistic similarity. At inference, c_s is formed by encoding each of the $k = 5$ reference images and averaging the embeddings before projecting to the model dimension.

One property of this encoder is central to how we read our results. Because it is trained by contrasting writing styles under a metric-learning objective, the representation it learns is organised around stroke-level style, the geometry of how letters are formed, rather than around the low-level ink-and-pen artefacts of any one sample. (The same encoder has been shown to carry its style representation across scripts, which is only possible if it has latched onto script-independent stroke properties rather than ink characteristics.) So the images DiffusionPen produces reproduce a writer’s style faithfully but render it in generic ink and contrast, without the specific ink, pen, and acquisition signature of that writer’s real samples. The downstream consequence, that synthetic data helps style-reliant (binarised) models but not artefact-exploiting (grayscale) models, is what Chapters 5 and 6 examine.

The denoising network is a UNet, conditioned at every step on both c_s and c_τ through cross-attention. In training it learns to predict the Gaussian noise added to a latent z_t at timestep t , minimising the mean-squared denoising loss $\|\epsilon - \epsilon_\theta(z_t, c_s, c_\tau)\|^2$. At sampling time a DDIM scheduler (Section 3.1.2) denoises pure Gaussian noise into a clean latent in 50 reverse steps, and the VAE decoder turns that latent into a word image.

3.4 Synthetic Data Generation

Once DiffusionPen has been trained on a dataset’s real training split, we generate synthetic word images per writer as follows. For each sample to be produced for a target writer w , five of w ’s word images are picked at random to set the style condition. The target transcription is taken from another writer’s word image, drawn from the training split. Sampling text this way keeps the generated words inside the dataset’s vocabulary while adding lexical variety.

The volume of synthetic data is stated relative to the real training set, per writer: “100% synthetic data” means generating, for each writer, as many synthetic images as that writer has real training images, and “200%” means twice as many. Fixing the ratio per writer keeps the synthetic-to-real balance steady no matter how much real data a given writer contributes. The generated images then feed the GR-RNN model in the synthetic-only and augmentation settings of Section 3.2.

Five style references are used per sample, following DiffusionPen’s few-shot design. Sampling uses the DDIM scheduler with fifty reverse steps, trading a little fidelity for a large saving in generation time, which matters when the synthetic set is as large as the real training data. On CVL, where the generation format is itself under study, a separate diffusion model is trained for each representation so that the format never confounds the comparison; on the historical data a single generator is used. These settings, with the full training configuration, are collected in Chapter 4.

Figures 3.3 and 3.4 give a qualitative sense of what the generator produces. For the same writer and the same target word, the generated images reproduce that writer’s handwriting style, the shape and slant of the strokes, while rendering it in generic ink, both on contemporary CVL handwriting (Figure 3.3) and on the historical BullingerDB material introduced in Section 3.7 (Figure 3.4).



Figure 3.3: Real (left) and DiffusionPen-generated (right) CVL word images for the same writer and the same transcription. The generated samples preserve each writer’s stroke-level style (for example writer 0244’s rounded hand, writer 0290’s narrow cursive, and writer 0291’s heavier, more angular strokes) while rendering it in generic ink rather than the writer’s original ink and pen.

3.5 Writer Identification Model: GR-RNN

The downstream writer model is GR-RNN (He and Schomaker, 2021), shown in Figure 3.5. We keep the authors’ original architecture and change only one thing: we also expose an embedding for retrieval (Section 3.6).

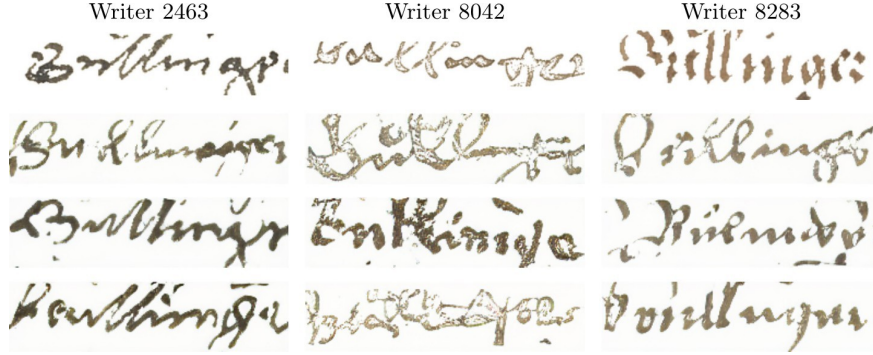


Figure 3.4: Real handwriting (top row) and DiffusionPen-generated samples (rows 2 to 4) for three BullingerDB writers. Even on degraded historical script, the generated samples follow each writer’s style. Reproduced from our earlier workshop paper.

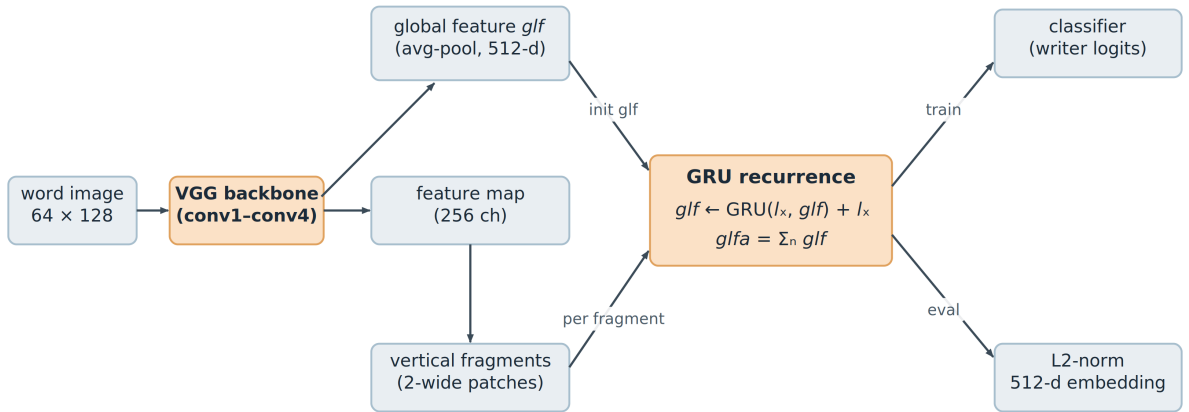


Figure 3.5: GR-RNN architecture. A VGG backbone produces a global feature and an intermediate feature map; the feature map is split into vertical fragments that are processed recurrently by a residual GRU, whose accumulated state $glfa$ is passed to a writer classifier during training and L2-normalised into a 512-dimensional embedding at evaluation time.

Within a word, GR-RNN’s combination of local fragments and a recurrent global context is well matched to writer identity, which shows up both in how individual strokes are shaped and in regularities that persist across a word. We deliberately leave the architecture unchanged from the original, so that any difference in performance can be attributed to the training data rather than to changes in the model, which is precisely what a study of synthetic data requires.

The backbone is a VGG-style network with four convolutional stages (channel widths 64, 128, 256, 512), each followed by max-pooling, that processes the word image. Two tensors are read off: a global feature *glf* from global average pooling of the final stage (512-dimensional), and an intermediate feature map of 256 channels taken before the last stage, which still holds spatial structure.

The feature map is broken into a sequence of local fragments. In the vertical mode we use throughout, it is split along its width into adjacent two-column patches. Each fragment is average-pooled, projected from 256 to 512 dimensions by a linear layer, and passed to a gated recurrent unit whose hidden state starts from the global feature *glf*. A residual connection adds the projected fragment back onto the GRU output, so every step keeps both the cross-fragment context built up in the recurrent state and the fragment’s own local content; the per-step states are summed into an accumulated representation *glfa*. This is how GR-RNN brings global context and local fragment detail together.

In training, *glfa* goes to a linear classifier with one output per training writer, and the network is optimised with a label-smoothing cross-entropy loss, which softens the one-hot targets and tends to improve generalisation and calibration. The model is trained as a closed-set classifier over the training writers; at test time, for retrieval, that classifier is set aside.

3.6 Retrieval Adaptation

For retrieval we run the same trained network in evaluation mode, where it returns the accumulated representation *glfa* after L2 normalisation instead of class logits, giving a 512-dimensional unit-norm embedding for which cosine similarity is just the dot product. Since the classifier head is unused at test time, the embedding applies perfectly well to writers never seen in training, as the open-set setting demands.

We evaluate retrieval by leave-one-out nearest-neighbour search, at two levels of granularity. At the word level, each test word image takes a turn as the query, with all other test word images as the gallery. An item counts as relevant if it shares the query’s writer. For a query with R relevant items in a ranking of the gallery, the average precision is

$$\text{AP} = \frac{1}{R} \sum_k P(k) \text{rel}(k), \quad (3.4)$$

where $\text{rel}(k) = 1$ when the item at rank k is relevant and $P(k)$ is the precision over the top k . The mean average precision (mAP) averages AP over all queries. We also report Top- k , the fraction of queries for which at least one same-writer item shows up in the top k , for $k \in \{1, 5, 10\}$.

At the document level, the unit of retrieval is the whole document rather than the single word. Word-level retrieval judges each word on its own, so we also build a per-document signature by

averaging the embeddings of every word image in a document and re-normalising. Documents are read off the image filename, which follows the format `writerid-documentnumber-linenummer-wordnumber-transcription`. The same leave-one-out mAP and Top- k are then computed over documents instead of words. Because averaging cancels out a lot of per-word noise, document-level retrieval is much stronger than word-level, as Chapter 6 reports.

3.7 Datasets and Splits

Our experiments use two datasets, summarised in Table 3.1.

CVL (Kleber et al., 2013) is a contemporary database of modern handwriting. It comprises 310 writers and 99,904 word images, collected under controlled conditions from volunteers who copied a set of printed texts, so the material is clean and evenly sampled across writers, and a natural starting point before the harder historical setting.

BullingerDB (Peer et al., 2026) is a large historical collection. The full corpus holds 796 writers and over four million word images, written across a long span during which a single scribe’s own hand may drift, which makes it hard for both identification and synthesis. Because of its size, we use only a subset, denoted SBullinger, built by dropping the 200 writers with the most images, which leaves 596 writers and 452,640 word images. Even after this trimming the subset stays highly uneven, and for many writers genuinely data-scarce: the number of images per writer ranges from 36 to 2,659. Together with the degradation and the within-writer drift of historical hands, this makes SBullinger considerably more challenging than the contemporary, evenly sampled CVL, despite its larger overall size.

The two tasks use different splits, reflecting their closed-set and open-set evaluation. For identification (closed-set), the same writers appear in training and test: the split is taken per writer, with 80% of each writer’s images forming the training set and the remaining 20% the test set (Table 3.1). For retrieval (open-set), the training and test writer populations are kept disjoint, so the embedding is judged on writers never seen in training (Table 3.2). On CVL, 27 writers (12,147 images) are used for training and the other 283 writers (87,757 images) for the retrieval test set. On SBullinger, 50 writers (43,404 images) train the model and 546 writers (409,236 images) test it.

Table 3.1: Datasets and identification split (80/20 per writer).

Dataset	Writers	Total images	Training set	Test set
CVL	310	99,904	79,790	20,114
SBullinger	596	452,640	362,114	90,526

Table 3.2: Retrieval data splits. Training and test writers are disjoint (open-set).

Dataset	Total writers	Total images	Training set	Test set
CVL	310	99,904	27 writers / 12,147 images	283 writers / 87,757 images
SBullinger	596	452,640	50 writers / 43,404 images	546 writers / 409,236 images

Chapter 4

Experimental Setup

This chapter describes the experimental configuration: preprocessing, the training settings for each model, the evaluation metrics, and the steps taken for reproducibility.

4.1 Preprocessing

Every word image is converted to the representation required by the respective experiment (RGB, grayscale, or binarised) and resized onto a fixed 64×128 pixel canvas (height $\times$ width), scaling to fit so that aspect ratio is preserved and the result is placed on the canvas. Where binarisation is used it follows Sauvola’s adaptive thresholding (Sauvola and Pietikäinen, 2000), with ink and background mapped to the two binary levels. Real and synthetic images go through exactly the same preprocessing, so the downstream model always sees a consistent input distribution, and the same preprocessing is used for training DiffusionPen and for training GR-RNN. On CVL we make use of all three representations: DiffusionPen is trained to generate in RGB, grayscale, and binarised form, so that the effect of the generation format can be measured, while GR-RNN is trained on grayscale or binarised images.

Two details of this preprocessing matter for the study. Resizing every word onto a common canvas while preserving aspect ratio gives the network a fixed input geometry without distorting letter proportions, which are themselves part of the writer signal. And applying the identical pipeline to real and synthetic images ensures that the downstream model cannot separate the two on the basis of formatting alone, so that any measured effect of synthetic data reflects the content of the generated handwriting rather than an incidental difference in how the images were prepared.

4.2 Training Configurations

DiffusionPen is trained in two stages: the style encoder is trained first, on its own, and the denoising (diffusion) model is then trained with the style encoder frozen, after which the GR-RNN writer model is trained separately. The latent autoencoder and the Canine-C text encoder are pre-trained and are not trained by us; only the style encoder, the diffusion model, and GR-RNN are trained here. The settings differ between the contemporary (CVL) and historical (SBullinger) datasets, reflecting their different sizes and difficulty, and are spelled out below.

4.2.1 Style Encoder

On CVL, the style encoder is trained for 100 epochs at batch size 320, with Adam at a learning rate of 10^{-3} (cut by a factor of 0.1 every three epochs) and a weight decay of 10^{-4} . On SBullinger, it is trained for 100 epochs at batch size 64, again with Adam but at a learning rate of 10^{-4} on a ReduceLROnPlateau schedule that halves the rate after five epochs without improvement in validation loss, a weight decay of 10^{-4} , and a dropout of 0.3 after the global average pooling layer to curb overfitting on the historical data. Negatives for the triplet loss are drawn at random; writers vary enough that random negatives are already hard enough, and we saw no convergence trouble.

4.2.2 Diffusion Model

The diffusion model is fine-tuned from the publicly released DiffusionPen checkpoint trained on IAM, with the frozen style encoder supplying the writer-style condition. On CVL it is fine-tuned for 500 epochs at batch size 640; on SBullinger for 100 epochs at batch size 1024. For both, the diffusion process uses 1000 timesteps in training while the DDIM scheduler drops the reverse steps to 50 at sampling time; the optimiser is AdamW at a learning rate of 10^{-4} with a weight decay of 0.2. On CVL we train a separate diffusion model for each input representation (RGB, grayscale, binarised), so that the effect of the generation format can be studied on its own.

4.2.3 Writer Identification Model

GR-RNN is implemented in PyTorch and trained with Adam at a learning rate of 10^{-4} and a weight decay of 10^{-4} , the rate halving every 10 epochs. The mini-batch is 16 on CVL and 64 on SBullinger. Models are trained for up to 150 epochs to confirm convergence, and all reported results are taken at epoch 50, by which point accuracy has plateaued (Section 4.3).

4.3 Training Dynamics

All identification results in this thesis are reported at epoch 50. To confirm that this is a fair operating point, we continue training on CVL to 150 epochs and track Top-1 accuracy throughout. Figures 4.1 and 4.2 show the curves for grayscale and binarised input respectively, under the real-only, synthetic-only (100% generated), and augmentation (real together with 100% generated) conditions.

For both representations the same picture emerges. Accuracy rises steeply over the first ten to twenty epochs and then flattens: beyond roughly epoch 30 the curves are almost level, and the values at epoch 50 sit within a fraction of a point of those at epoch 150. The ordering of the three conditions is settled just as early and does not change thereafter, with augmentation and real-only close together and synthetic-only well below them. Reporting at epoch 50 therefore captures converged performance while avoiding the cost of training every configuration to 150 epochs.

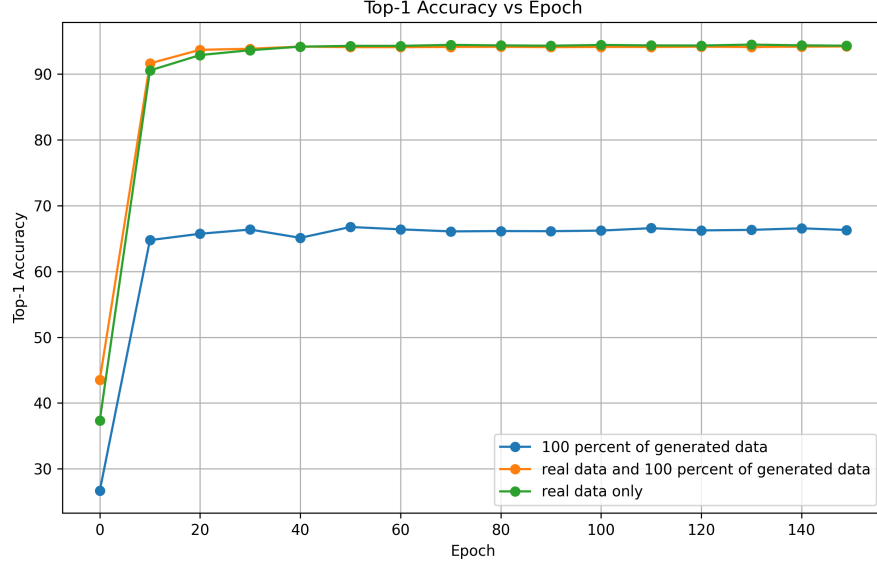


Figure 4.1: CVL Top-1 accuracy over training, grayscale input, for the real-only, synthetic-only (100% generated), and augmentation (real + 100% generated) conditions. Accuracy plateaus well before epoch 50, the point at which all results are reported, and stays consistent to epoch 150.

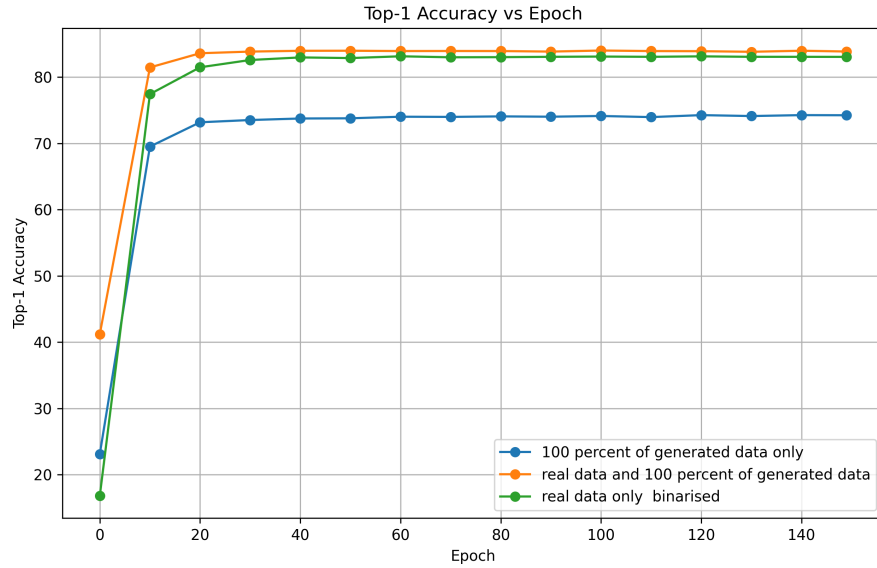


Figure 4.2: CVL Top-1 accuracy over training, binarised input, for the same three conditions. As with grayscale input, accuracy converges before epoch 50 and remains stable through epoch 150.

4.4 Evaluation Metrics

For identification, we report Top-1 and Top-5 classification accuracy on each dataset’s fixed real test set. Top-1 accuracy is the fraction of test word images whose predicted writer, the class with the highest logit, is correct. Top-5 accuracy is the fraction for which the true writer appears among the five highest-scoring classes. Top-5 is the more forgiving of the two and is informative when several writers share visually similar hands, since it credits the model for placing the correct writer near the top of its ranking even when that writer is not ranked first.

Retrieval is evaluated as a ranking problem rather than a classification problem, because the test writers are disjoint from the training writers and have no learned class of their own (Section 3.7). Each test item is used in turn as a query, and the remaining test items form the gallery, ranked by increasing cosine distance between their L2-normalised embeddings; a gallery item is relevant to the query when it was written by the same writer. We summarise the resulting rankings with mean average precision and Top- k .

For a single query q with R_q relevant items in a gallery of size N , the average precision is the mean of the precision values taken at each rank at which a relevant item is retrieved,

$$\text{AP}(q) = \frac{1}{R_q} \sum_{k=1}^N P(k) \text{rel}(k), \quad (4.1)$$

where $P(k)$ is the precision at cut-off k (the fraction of the top k retrieved items that are relevant) and $\text{rel}(k) = 1$ if the item at rank k shares the query’s writer and 0 otherwise. Mean average precision is the average of $\text{AP}(q)$ over all queries, and because it integrates precision across the whole ranked list it rewards a method for placing same-writer items high consistently rather than only at the head of the list.

Top- k is the fraction of queries for which at least one same-writer item appears among the top k retrieved results, reported for $k \in \{1, 5, 10\}$. It is a coarser and more operational measure: Top-1 asks whether the single nearest neighbour shares the writer, while Top-5 and Top-10 ask whether a same-writer item lies within a short candidate list that a human could quickly check. Both metrics are computed at the word level, where each individual word image is a query, and at the document level, where the query and gallery items are the mean, re-normalised embeddings of whole documents (Section 3.6).

4.5 Reproducibility

For reproducibility, random seeds are fixed across Python, NumPy, and PyTorch (CPU and GPU), cuDNN is put in deterministic mode with auto-tuning off, and the data loaders use seeded workers and a seeded shuffling generator. Image lists and writer-identity orderings are sorted, so dataset construction is deterministic from run to run. To help others reproduce and build on the work, we release our train/test splits and the fine-tuned DiffusionPen checkpoints.

Chapter 5

Writer Identification Results

This chapter reports the writer identification results. We begin with a direct test of how faithfully DiffusionPen preserves a writer’s style (Section 5.1), which frames the experiments that follow, and then examine the effect of image representation on CVL (Section 5.2). From there we look at the effect of synthetic-data volume (RQ1.2, Section 5.3) and at what happens as real data becomes scarce, in the low-resource regime (RQ2.1, Section 5.4). Section 5.5 moves to the historical SBullinger dataset (RQ3.1), Section 5.6 checks robustness to the random seed, and Section 5.7 steps back to discuss why the pattern looks the way it does. Unless noted otherwise, the numbers are Top-1 accuracy at epoch 50 on the fixed real test set.

5.1 Style Preservation

A premise underlying this chapter is that DiffusionPen reproduces a writer’s stroke-level style but not the incidental ink-and-pen characteristics of their real samples. This can be measured directly, before we turn to the identification experiments that build on it. We take a GR-RNN trained on real CVL data and use it to classify the generated images, asking how often a writer’s synthetic samples are recognised as that writer, a train-on-real, test-on-synthetic protocol. A high accuracy means the generated images carry the writer signal the real-trained model has learned; a low one means they do not. Table 5.1 reports this for both representations, with the generated set sized at 100% and 200% of the real training data.

Table 5.1: Style preservation on CVL. A GR-RNN trained on real data classifies the generated images; accuracy (%) is how often a writer’s synthetic samples are attributed to that writer.

Image type	Generated ratio	Top-1	Top-5
Grayscale	100%	9.17	22.67
Grayscale	200%	5.03	16.05
Binarised	100%	78.19	91.85
Binarised	200%	78.08	91.68

The contrast between the two representations is large and central to the thesis. A real-trained binarised model recognises the generated samples as their target writer about 78% of the time, not far below its 82.87% accuracy on real binarised data, so on stroke shape alone the synthetic images are faithful to the writer. A real-trained grayscale model, by contrast, recognises

them only about 9% of the time, against 94.29% on real grayscale data. To rule out that this low grayscale attribution merely reflects a background mismatch, we whiten the generated backgrounds (non-ink pixels set to white via the Sauvola mask) and re-evaluate: attribution rises only from 9.17% to 19.02% at 100% and 5.03% to 15.88% at 200%, still far below the 94.29% real-word baseline, so the missing signal is the writer’s real ink rather than the background. This gap indicates that the grayscale model recognises writers partly from the patterns of the writing utensil and ink, cues tied to a writer’s real samples that the generator, rendering in generic ink, does not reproduce. The generated images therefore preserve the writer’s stroke-level style, which is all a binarised model sees, but not the ink-and-pen characteristics that a grayscale model also relies on: rendered in generic ink, a synthetic sample no longer matches the real grayscale appearance of its writer. The figures are nearly flat between the 100% and 200% volumes, as expected, since style preservation is a property of the generator rather than of how many samples are drawn. This is the most direct evidence for the pattern that recurs throughout the chapter, that synthetic augmentation helps binarised models and not grayscale ones, and it is taken up again in the discussion.

5.2 Effect of Image Representation

Table 5.2 gives CVL accuracy for every pairing of the format DiffusionPen is trained on with the format GR-RNN consumes, under three conditions: generated data only, real data only, and real data combined with 100% generated data.

Table 5.2: CVL Top-1 accuracy (%) for combinations of DiffusionPen training format and GR-RNN input format.

DiffusionPen	GR-RNN	Gen. only	Real only	100% Gen. + Real
RGB	Grayscale	66.76	94.29	94.09
RGB	Binarised	73.78	82.87	83.98
Grayscale	Grayscale	49.84	94.29	89.96
Grayscale	Binarised	50.75	82.87	80.23
Binarised	Grayscale	59.56	87.17	84.47
Binarised	Binarised	40.50	82.87	83.65

Two things are visible in the table. The first concerns the real-only baseline, which does not involve DiffusionPen and so depends only on the format GR-RNN takes: grayscale input gives a higher baseline than binarised input, roughly 94% against 83%. This difference is a property of the recognition model and the data rather than of the synthetic-data question, and its interpretation is taken up in Section 5.7; for present purposes it simply fixes two operating points against which synthetic data is measured.

The second, and the one of interest here, is that the effect of synthetic data is governed by the format the model consumes rather than by its absolute accuracy. Adding generated data raises accuracy only when GR-RNN works on binarised images and the generator was trained on RGB, where the real-only result of 82.87% rises to 83.98%. In every grayscale configuration the same addition leaves accuracy unchanged or lowers it, and synthetic data from a grayscale-trained generator is harmful throughout. The generation format thus has a marked influence on

usefulness: an RGB-trained DiffusionPen yields the most useful samples and a grayscale-trained one the least, so all subsequent experiments use the RGB-trained generator.

5.3 Effect of Synthetic-Data Volume

Tables 5.3 and 5.4 report Top-1 and Top-5 accuracy for the RGB-trained DiffusionPen as the amount of synthetic data is raised to 100% and 200% of the real training set, for grayscale and binarised input respectively.

Table 5.3: Effect of synthetic-data volume on CVL grayscale accuracy (%), RGB-trained DiffusionPen.

Training data	Top-1	Top-5
Real only	94.29	98.66
100% Gen. + Real	94.09	98.70
200% Gen. + Real	93.97	98.67
100% Gen. only	66.76	84.37
200% Gen. only	66.57	84.43

Table 5.4: Effect of synthetic-data volume on CVL binarised accuracy (%), RGB-trained DiffusionPen.

Training data	Top-1	Top-5
Real only	82.87	93.98
100% Gen. + Real	83.98	94.65
200% Gen. + Real	84.07	94.97
100% Gen. only	73.78	89.08
200% Gen. only	78.99	92.12

The two representations respond to volume in opposite ways, along the split already seen in Section 5.2. For binarised input, accuracy rises as synthetic data is added, from a real-only Top-1 of 82.87% to 84.07% once twice the training volume is supplied, with the synthetic-only and Top-5 figures moving in the same direction; across the range tabulated here the benefit increases with volume. For grayscale input the additional data brings no improvement, mixed accuracy staying within a fraction of a point of its real-only value and the synthetic-only figures remaining flat. Volume therefore helps precisely where synthetic data helps at all, and does nothing where it does not.

Tables 5.3 and 5.4 stop at twice the real data. Pushing the binarised augmentation further, with the generator still trained on the full real set, shows that the benefit does not grow without bound. Figure 5.1 tracks Top-1 accuracy as the amount of generated data is raised from 10% to 500% of the real training set, for the synthetic-only and augmentation conditions against the real-only baseline. Both conditions improve up to about 200% and then turn down: augmentation peaks at 200% and afterwards falls back, dropping below the real-only baseline of 82.87% by 400% (82.14%) and to 81.53% at 500%, while the synthetic-only condition peaks at the same point (78.99%) before drifting down to 75.94%. Past a moderate multiple of the real data, then, additional synthetic samples begin to dilute rather than reinforce the real signal, so there is a sweet spot, here around 200%, rather than a monotone gain. Even with a generator trained on

the full real set, indiscriminately adding more synthetic data is counterproductive beyond that point.

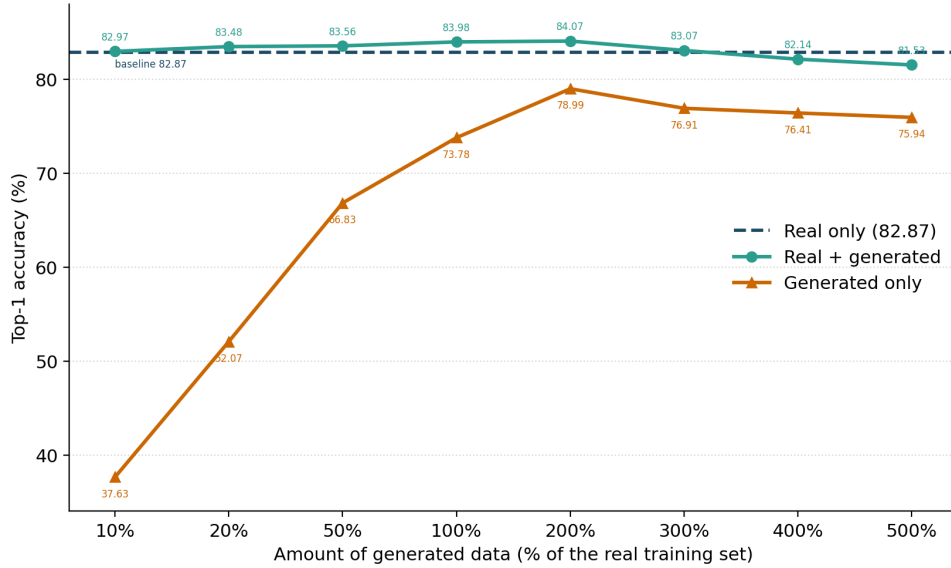


Figure 5.1: CVL Top-1 accuracy (binarised) as the amount of generated data grows from 10% to 500% of the real training set, with the generator trained on the full real data. Both the synthetic-only and the augmentation conditions peak around 200% and decline beyond it, with augmentation falling below the real-only baseline by 400%.

5.4 Effect of Real-Data Availability

To probe the low-resource regime, we cut the real data used to train both DiffusionPen and GR-RNN down to a fraction of the full training set, while keeping the synthetic pool fixed at the full training-set size (about 79,790 images). Everything in this section uses binarised input. Figure 5.2 plots Top-1 accuracy for the real-only, generated-only, and mixed conditions as that fraction runs from 10% to 100%.

Three observations follow from the curve. First, the mixed condition stays above the real-only baseline at every fraction, and the margin is largest where real data is scarcest: at a tenth of the training set it is about seven and a half points (41.20% against 33.66%), narrowing to about a point at full data. Augmentation is thus worth most exactly when labelled data is in short supply, the regime that motivates the thesis. Second, at the smallest fractions the synthetic-only model also overtakes the real-only one, so when very little real data is available the generated data alone is preferable to it; beyond roughly a third of the training set the synthetic-only curve levels off and falls behind the real baseline. Third, that synthetic-only curve climbs steeply as the real fraction grows even though the amount of synthetic data is held fixed: the same number of images yields 35.88% when the generator saw a tenth of the data and 73.78% when it saw all of it. This separates the quality of the generator from the sheer quantity of its output, and shows that the usefulness of synthetic data depends on how much real data the generator itself was trained on (RQ2.2).

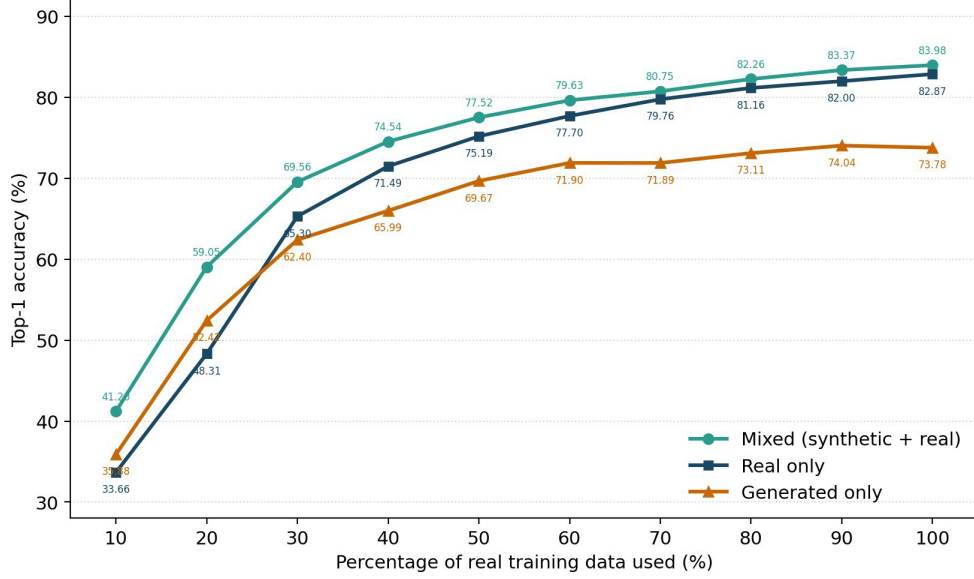


Figure 5.2: CVL Top-1 accuracy (binarised) as the fraction of real training data varies. The real-only and mixed conditions train GR-RNN on that fraction of real data (with the synthetic pool held constant for the mixed condition), while the generated-only condition trains on a fixed-size synthetic pool produced by a DiffusionPen trained on the same fraction.

5.5 Historical Handwriting: SBullinger

Table 5.5 gives identification accuracy on the historical SBullinger dataset.

Table 5.5: SBullinger Top-1 accuracy (%) by GR-RNN input format.

GR-RNN	Gen. only	Real only	100% Gen. + Real
Grayscale	25.72	88.04	85.80
Binarised	44.83	77.29	78.62

The contemporary pattern reappears. Synthetic augmentation again improves binarised identification (from 77.29% to 78.62%) and again lowers grayscale identification (from 88.04% to 85.80%), the gains and losses smaller than on CVL and the absolute accuracies lower, as expected for degraded historical material in which a single hand may drift over time. The synthetic-only results point the same way, far stronger for binarised than for grayscale input (44.83% against 25.72%), consistent with synthetic data carrying the stroke-shape information that binarised models depend on while lacking the ink-and-pen cues that grayscale models exploit. That the direction of the effect survives the move from a clean modern corpus to a noisy historical one is the substantive finding here.

5.6 Robustness to Random Seed

All results reported elsewhere in this thesis use a single random seed (42). To check that they are not an artefact of that choice, we re-ran the main CVL identification experiments, the 100% conditions for both representations, under two further seeds (123 and 999). Table 5.6 reports per-seed Top-1 accuracy together with the mean and standard deviation over the three seeds.

Table 5.6: CVL Top-1 accuracy (%) across three random seeds, with mean and sample standard deviation.

Input	Condition	Seed 42	Seed 123	Seed 999	Mean	Std
Binarised	Gen. only	73.78	74.07	73.85	73.90	0.15
Binarised	Gen. + Real	83.98	83.99	83.74	83.90	0.14
Binarised	Real only	82.87	82.90	82.71	82.83	0.10
Grayscale	Gen. only	66.76	64.91	66.32	66.00	0.97
Grayscale	Gen. + Real	94.09	94.21	93.94	94.08	0.14
Grayscale	Real only	94.29	93.94	94.30	94.18	0.21

The spread across seeds is small. For every condition except grayscale generated-only the standard deviation is at or below 0.21 points, so the single-seed numbers reported elsewhere are representative to within a few tenths of a point. What matters more is that the variation is small relative to the effects the conclusions rest on. The binarised augmentation gain, a mean of 83.90% against a real-only mean of 82.83%, is about a point, several times the corresponding seed-to-seed standard deviations of roughly 0.1, so it is a real effect rather than seed noise. The grayscale augmentation change, by contrast, a mean of 94.08% against a real-only mean of 94.18%, is about a tenth of a point and sits within the seed variation, which is consistent with synthetic augmentation being genuinely neutral for grayscale input. The one condition with appreciable spread is grayscale generated-only (standard deviation 0.97), where training on synthetic data alone is somewhat more sensitive to the seed, though still far below the real-only and augmentation levels. The seed analysis therefore reinforces the central identification finding: the binarised gain holds across seeds, and the grayscale neutrality is not an accident of a single run.

5.7 Discussion

Across both datasets, both volumes, and the low-resource regime, the results converge on a single pattern: synthetic augmentation helps models that consume binarised images and does not help, sometimes hurting, models that consume grayscale images. This is the behaviour anticipated in Chapter 3 from the design of DiffusionPen’s style encoder.

The style encoder is trained by contrasting writing styles under a metric-learning objective, so it captures stroke-level style but not the low-level ink, pen, and acquisition characteristics of individual samples. The synthetic images it conditions therefore reproduce a writer’s style faithfully while rendering it in generic ink and contrast; the qualitative samples in Figures 3.3 and 3.4 show this directly, the generated words following each writer’s stroke shapes but carrying generic ink. A binarised model has already discarded ink intensity and must work from stroke shape, which is exactly what the synthetic data supplies, so additional synthetic samples reinforce the signal it relies on and accuracy grows with volume. A grayscale model, by contrast, can also draw on ink-and-pen regularities that happen to correlate with writer identity in the real data; because the synthetic data does not reproduce those regularities, adding it dilutes rather than strengthens the signal, and accuracy stays flat or drifts slightly down.

This reading also accounts for an apparent paradox, that grayscale models reach the higher

absolute accuracy on CVL and yet benefit least from synthetic data. Part of that accuracy rests on ink-and-pen regularities that are informative within a single dataset but are not intrinsic to handwriting style, and that synthetic data cannot supply; a model leaning on them may transfer less well to other collections or acquisition conditions, even as it scores well on a fixed test set. Seen through the lens of writer style, the binarised setting, the one in which synthetic augmentation actually helps, is the more demanding of the two, since it requires the model to identify writers from how they form their strokes rather than from which pen or scanner produced the image. Synthetic handwriting is therefore most valuable precisely where identification is forced to rely on style, the thread that runs through every result in this chapter.

A practical reading follows from this. When the downstream model is confined to stroke shape, as a binarised model is, diffusion-generated handwriting is worth adding, and most worth adding when real labelled data is scarce; when the model also consumes ink intensity, as a grayscale model does, synthetic data of this kind brings little and can mildly hurt. The deciding factor is not the generator’s raw image quality but the match between what the generator reproduces faithfully, namely style, and what the downstream representation is able to use. This reframes the question a practitioner should ask before generating data: not simply whether the synthetic handwriting looks realistic, but whether the model depends on the part of the signal that the generator gets right.

The generation format carries a related and equally practical lesson. The most useful synthetic data on CVL came from a DiffusionPen trained to generate full RGB images, even though the downstream models never consume RGB, while a generator trained directly on binarised images produced the least useful data of all. The likely reason is that learning to reproduce the richer RGB signal forces the generator to model handwriting more completely, after which the conversion to grayscale or binarised discards only the incidental ink-and-pen information; a generator trained on already-binarised images never has to capture that fuller structure in the first place. In practice this argues for training the generator on the richest available representation and leaving the conversion to the downstream format to preprocessing, rather than training the generator directly in the format the recogniser will use.

Chapter 6

Writer Retrieval Results

This chapter reports open-set writer retrieval, where the training and test writers are disjoint (Section 3.7). We contrast word-level and document-level retrieval on CVL and look at what synthetic data does (RQ1.1, RQ3.2), consider the role of image representation, turn to historical SBullinger retrieval (RQ3.1), and close with a discussion. Every retrieval result uses the RGB-trained DiffusionPen for synthesis and reports mAP and Top- k as defined in Section 4.4.

6.1 Word-Level versus Document-Level Retrieval on CVL

Tables 6.1 and 6.2 report CVL retrieval at the word and document levels, for grayscale and binarised representations, under the generated-only, real-only, and mixed conditions.

Table 6.1: CVL word-level retrieval (%). Each word image is a query against all other test words.

Format	Condition	mAP	Top-1	Top-5	Top-10
Grayscale	Real only	10.68	51.13	75.59	83.36
Grayscale	Gen. + Real	10.49	49.27	73.70	81.81
Grayscale	Gen. only	8.29	46.55	71.60	80.19
Binarised	Real only	4.82	29.66	54.57	65.53
Binarised	Gen. + Real	5.03	29.34	53.55	64.50
Binarised	Gen. only	4.48	28.25	52.80	63.90

Table 6.2: CVL document-level retrieval (%). Each query is the mean, re-normalised embedding of a document’s words.

Format	Condition	mAP	Top-1	Top-5	Top-10
Grayscale	Real only	89.24	96.17	97.87	98.37
Grayscale	Gen. + Real	90.58	96.74	98.23	98.51
Grayscale	Gen. only	88.07	95.81	97.87	98.23
Binarised	Real only	86.02	94.75	98.01	98.51
Binarised	Gen. + Real	88.12	96.10	98.37	98.65
Binarised	Gen. only	85.19	94.54	97.80	98.44

The most striking feature of the two tables is the difference between the granularities. Word-level mAP is low, from roughly 5% for binarised to about 11% for grayscale, whereas document-

level mAP is high, between 85% and 91%. This reflects the difficulty of the task rather than a failure of the model: retrieving same-writer words from a gallery of tens of thousands of words written by 283 unseen writers is intrinsically hard. Even so, word-level Top-1 of 29 to 51% lies far above the sub-percent level expected from random ranking, so the embeddings clearly carry writer information at the word level. Averaging a document’s word embeddings into a single signature cancels much of the per-word noise and produces the strong document-level figures that constitute the practical retrieval result. Document-level evaluation is therefore the level at which retrieval should be judged, and aggregation is what makes it work (RQ3.2). The size of this word-to-document gap is quantitatively in line with the text-quantity study of Peer et al. (2025), who report that word- and line-level retrieval on CVL falls well short of page-level performance.



Figure 6.1: An example of word-level writer retrieval on CVL, showing how retrieval works in practice. Each query word image on the left is compared against the gallery of all other test words, and its ten nearest neighbours are shown in rank order, bordered solid green when written by the same writer and dashed red otherwise; the count of same-writer matches among the ten is given at the right. The three queries illustrate the range of outcomes the embedding produces, from one whose ten nearest neighbours are all the same writer to ones with only a few.

What synthetic data contributes depends on the granularity. At the document level it consistently helps, raising mAP for both representations (from 86.02% to 88.12% for binarised and from 89.24% to 90.58% for grayscale) with matching improvements in Top-1. At the word level it is close to neutral, marginally lower for grayscale and marginally higher for binarised. The synthetic-only condition is revealing: at the document level it almost matches the real-only baseline (88.07% against 89.24% for grayscale, 85.19% against 86.02% for binarised), an indication of how much of the writer signal the synthetic data carries on its own, and when combined with real data it augments the baseline (RQ1.1).

One contrast with identification is worth pulling out. In identification, synthetic data helped only the binarised models; here, at the document level, it helps the grayscale models too. The likely reason is that open-set retrieval cannot fall back on the ink-and-pen shortcuts available to closed-set grayscale identification, because the test writers and documents are unseen. The embedding has to rely on generalisable style, and the extra stylistic variety that synthetic data brings improves that generalisation whatever the representation.

6.2 Effect of Representation in Retrieval

The representation hierarchy seen in identification carries over to retrieval. Grayscale embeddings are stronger than binarised ones at both granularities, by more than a factor of two at the word level and by a few points at the document level. Aggregation narrows that gap and synthetic augmentation narrows it further, bringing binarised document-level mAP to within about two points of the grayscale figure. As in identification, the representation with the lower baseline is the one that gains most from synthetic data, while the absolute ordering of the representations is a property of the data rather than of the synthetic-data question.

6.3 Historical Handwriting: SBullinger

Tables 6.3 and 6.4 report retrieval on the historical SBullinger dataset at the word and document levels, for grayscale and binarised input, under the same three conditions used on CVL. For these experiments the generated set comprises 43,500 word images, 870 for each of the 50 training writers, matching the real training set in size.

Table 6.3: SBullinger word-level retrieval (%).

Format	Condition	mAP	Top-1	Top-5	Top-10
Grayscale	Real only	6.38	44.58	67.84	76.14
Grayscale	Gen. + Real	5.64	38.42	62.32	71.55
Grayscale	Gen. only	4.56	44.21	66.90	75.18
Binarised	Real only	3.33	33.09	56.63	66.41
Binarised	Gen. + Real	3.49	31.96	55.93	65.87
Binarised	Gen. only	2.86	31.83	54.98	64.81

Table 6.4: SBullinger document-level retrieval (%).

Format	Condition	mAP	Top-1	Top-5	Top-10
Grayscale	Real only	25.54	24.01	36.47	41.27
Grayscale	Gen. + Real	25.69	25.02	35.73	41.27
Grayscale	Gen. only	26.17	23.82	35.73	40.17
Binarised	Real only	32.87	30.10	40.81	44.69
Binarised	Gen. + Real	31.93	28.72	41.18	45.24
Binarised	Gen. only	32.29	29.92	39.70	44.23

Historical retrieval is markedly harder than contemporary retrieval. Word-level mAP sits in the low single digits and document-level mAP between roughly 25% and 33%, far below the corresponding CVL figures, which reflects the degradation of the material, the drift of a single writer’s hand over time, and the smaller number of documents per writer in this open-set setting. Aggregation remains essential, lifting mAP from the word level to the document level just as it does on CVL.

The two representations behave differently here than on CVL. At the word level grayscale is again the stronger of the two, but at the document level the order reverses: binarised aggregates to a higher mAP than grayscale (32.87% against 25.54% for real data), which suggests that on

degraded historical script the binarised representation yields document signatures that separate writers more cleanly. The effect of adding synthetic data on top of real data is small and mixed: for grayscale it is slightly negative at the word level and neutral at the document level, and for binarised it is marginally positive at the word level and marginally negative at the document level, so augmentation brings no consistent benefit on this material. More notable is the generated-only condition. Unlike on CVL, where synthetic-only word retrieval fell well behind real, here it is competitive: training on generated data alone comes close to real at rank one (44.21% against 44.58% for grayscale, 31.83% against 33.09% for binarised) and stays within a point of it at the document level, slightly above for grayscale (26.17% against 25.54%). The synthetic samples therefore carry most of the writer signal on their own, but the difficulty of the degraded material and the drift within a single writer’s hand mean the extra stylistic variety they add on top of real data does not translate into better open-set generalisation.

6.4 Discussion

The retrieval results confirm and extend the identification findings. First, document-level aggregation is decisive: it lifts CVL retrieval from single- or low-double-digit word-level mAP to between 85% and 91% at the document level, and it is the level at which writer retrieval should be assessed. Second, synthetic data improves document-level retrieval on CVL for both representations, and synthetic-only embeddings come close to real-only ones, showing that synthetic data is a workable source of training signal for retrieval and not merely a supplement to real data. Third, the representation hierarchy matches identification, with binarised retrieval gaining most from augmentation, and with one notable difference: in the open-set setting synthetic augmentation helps grayscale at the document level as well, consistent with the absence of ink-and-pen shortcuts when the test writers are unseen. Finally, historical retrieval remains the hardest setting: the synthetic augmentation that helped on CVL is at best neutral and at times slightly harmful on SBullinger, where the difficulty of the degraded material and the drift within a single writer’s hand dominate. A further wrinkle is that the representations swap order at the document level on SBullinger, with binarised aggregating to a higher mAP than grayscale, which underlines that the absolute ranking of representations is a property of the data rather than something fixed across collections.

Two practical points follow for anyone applying synthetic data to retrieval. First, retrieval should be reported and optimised at the document level: word-level matching is a useful internal building block, but it is too noisy to serve as the figure of merit, and the document signature is what carries writer identity in practice. Second, the value of synthetic augmentation tracks the same style-versus-artefact logic seen in identification, with the open-set twist that even grayscale benefits once the test writers are unseen, because the ink-and-pen shortcuts grayscale exploits within a closed set no longer transfer. Where neither condition is favourable, as on the degraded historical material, synthetic augmentation cannot be relied on to help and may slightly hurt, so on such collections it is best treated as something to validate rather than assume.

Chapter 7

Conclusion and Future Work

This thesis asked whether diffusion-generated synthetic word images can serve as training data for word-level writer identification and retrieval, using DiffusionPen as the generator and GR-RNN as the writer model, across a contemporary dataset (CVL) and a historical one (SBullinger). This chapter pulls together the answers to the research questions and points to future work.

7.1 Summary of Findings

The first research question (RQ1) asked to what extent, and in what way, diffusion-generated word images improve word-level identification and retrieval when added to real training data, through augmentation and through volume. Used as augmentation, synthetic data improves binarised identification on both datasets and improves document-level retrieval for both representations on CVL, while being neutral or slightly negative for grayscale identification and for word-level retrieval (RQ1.1). Even on their own the synthetic images carry a surprising amount of the writer signal, with synthetic-only training coming close to real-only for document-level retrieval embeddings. The benefit grows with synthetic volume in the binarised identification setting, where doubling the synthetic data lifted Top-1 accuracy from 82.87% (real only) to 84.07% (RQ1.2), though beyond roughly twice the real volume the extra data begins to dilute the training set rather than help.

The second research question (RQ2) asked how the usefulness of synthetic data depends on the amount of real data available, both to the downstream model and to the generator. The benefit is largest exactly where real data is scarcest: the gap between mixed and real-only identification was widest at 10% of the training data, a 7.5-point improvement, shrinking to about a point at full data, and under the most extreme scarcity synthetic-only identification actually beats real-only (RQ2.1). The usefulness of the generated data also depends on how much real data trained the generator: at a fixed synthetic volume, identification accuracy rose from 35.88% to 73.78% as the generator’s training fraction grew from 10% to 100% (RQ2.2).

The third research question (RQ3) asked how far this usefulness carries across domains, tasks, and retrieval granularities. It transfers from contemporary to historical handwriting (RQ3.1): on SBullinger, synthetic data again helped binarised identification and hurt grayscale identification, though the margins are smaller and the absolute accuracies lower, and historical retrieval is a good deal harder than contemporary retrieval. The tasks and granularities differ in a revealing

way (RQ3.2): in closed-set identification synthetic data helped only binarised models, but in open-set document-level retrieval it helped grayscale models too, in keeping with the absence of ink-and-pen shortcuts when the test writers are unseen; and across every retrieval experiment, document-level aggregation was decisive, lifting CVL retrieval from single-digit word-level mAP to roughly 90% at the document level.

Beneath all three questions sits image representation, which proved to be the single biggest factor in whether synthetic data helps. Synthetic augmentation helps models that consume binarised images and does not help, sometimes harms, models that consume grayscale images. The generation format matters as well: an RGB-trained DiffusionPen produces the most useful synthetic data, while a grayscale-trained one is harmful, and the value of the synthetic data also grows with the amount of real data used to train the generator, with identification accuracy rising from 35.88% to 73.78% as the generator’s training fraction grew from 10% to 100% at a fixed synthetic volume. We read all of this through handwriting style: DiffusionPen’s style encoder, trained by contrasting writing styles, captures stroke-level style but not the ink-and-pen artefacts of individual samples, so synthetic data reinforces the style signal that binarised models rely on while failing to supply the ink-and-pen cues that grayscale models exploit.

Taken together, these results show that diffusion-generated handwriting is a genuinely useful source of training data for writer identification and retrieval, most clearly in the low-resource regime and precisely when the model is held to writing style rather than to incidental ink-and-pen characteristics.

7.2 Future Work

Three directions stand out from this work. The first is to study how synthetic data behaves when the generator is applied to writers and data it has not seen. Here DiffusionPen was always trained on a dataset before generating, and synthetic images were produced only for writers present in that training data. Because the model conditions on a few style reference images, it can in principle generate for writers it never encountered during its own training, so a natural next step is to generate samples for unseen writers, or from a DiffusionPen trained on a different collection altogether, and to measure whether the augmentation benefit still holds. This would show how far the usefulness of synthetic data depends on the generator having already seen the target writers, and would line the generation side up with the open-set conditions that retrieval assumes.

The second direction is to take the same question beyond handwriting. The central finding here, that the value of synthetic training data is governed by how well the generator reproduces the part of the signal the downstream model relies on, is not specific to handwriting images. The same recipe, generating data with a modern conditional generative model and using it to train a recognition or retrieval model, could be tried on other tasks and on other modalities entirely, for example speaker identification and retrieval from audio, or person and action recognition from video, where labelled data is similarly scarce and where synthetic augmentation may well show the same dependence on representation.

A third direction is a systematic comparison of different handwriting generators. This study used DiffusionPen throughout, so its findings speak to that generator rather than to diffusion-

based synthesis in general. Repeating the analysis across a range of generators, from GAN-based models to other diffusion architectures, while holding the downstream identification and retrieval setup fixed, would show how much the usefulness of synthetic data depends on the particular generator, and whether the style-versus-artefact pattern found here is specific to DiffusionPen or a more general property of generated handwriting for writer identification and retrieval.

Chapter 8

Declaration on the Use of Generative AI

While working on this thesis I made use of generative AI tools, namely the AI chat assistants Claude and ChatGPT. I see these tools mainly as a way to learn, and that is how I used them for the most part. When there was a topic I did not yet understand, I would ask the assistant to explain it and to help me build up the background I needed, and I then went back to the original papers to check what I had learned. On the research side they were useful for getting an overview of the field: they helped me find current state-of-the-art work and pointed me towards researchers and papers working on the kind of problems I was looking into, which I could then follow up on my own. I also used them while writing, to rephrase sentences that were unclear, to improve the grammar and wording of my English, and to get a second opinion on how to structure some parts of the text.

I did not use these tools to produce the research itself. The research questions, the experiments, the results, and the conclusions are my own work, and every number and finding reported here comes from my own experiments and from the sources I cite. I read through and edited anything that was written with the help of these tools, and I take full responsibility for the content of this thesis.

References

- Alonso, E., Moysset, B., and Messina, R. (2019). Adversarial generation of handwritten text images conditioned on sequences. In *International Conference on Document Analysis and Recognition (ICDAR)*, pp. 481–486.
- Clark, J. H., Garrette, D., Turc, I., and Wieting, J. (2022). Canine: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics (TACL)*, 10:73–91.
- Davis, B., Tensmeyer, C., Price, B., Wigington, C., Morse, B., and Jain, R. (2020). Text and style conditioned GAN for the generation of offline handwriting lines. In *British Machine Vision Conference (BMVC)*.
- Fiel, S. and Sablatnig, R. (2015). Writer identification and retrieval using a convolutional neural network. In *International Conference on Computer Analysis of Images and Patterns (CAIP)*, pp. 26–37.
- Fogel, S., Averbuch-Elor, H., Cohen, S., Mazor, S., and Litman, R. (2020). ScrabbleGAN: Semi-supervised varying length handwritten text generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4324–4333.
- Gorges, et al. (2026). ICDAR 2026 competition on writer identification and pen classification from hand-drawn circles (CircleID). Accepted for presentation at the *International Conference on Document Analysis and Recognition (ICDAR)*.
- He, S. and Schomaker, L. (2020). FragNet: Writer identification using deep fragment networks. *IEEE Transactions on Information Forensics and Security (TIFS)*, 15:3013–3022.
- He, S. and Schomaker, L. (2021). GR-RNN: Global-context residual recurrent neural networks for writer identification. *Pattern Recognition*, 117:107975.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 6840–6851.
- Kang, L., Riba, P., Wang, Y., Rusiñol, M., Fornés, A., and Villegas, M. (2020). GANwriting: Content-conditioned generation of styled handwritten word images. In *European Conference on Computer Vision (ECCV)*, pp. 273–289.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.
- Kleber, F., Fiel, S., Diem, M., and Sablatnig, R. (2013). CVL-Database: An off-line database for writer retrieval, writer identification and word spotting. In *International Conference on*

- Document Analysis and Recognition (ICDAR)*, pp. 560–564.
- Koepf, M., Kleber, F., and Sablatnig, R. (2022). Writer identification and writer retrieval using vision transformers. In *Document Analysis Systems (DAS)*, pp. 352–366.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.
- Louloudis, G., Gatos, B., Stamatopoulos, N., and Papandreou, A. (2013). ICDAR 2013 competition on writer identification. In *International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1397–1401.
- Mayr, M., Dreier, M., Kordon, F., Seuret, M., Zöllner, J., Stutzmann, D., Mayr, A., and Christlein, V. (2025). Zero-shot paragraph-level handwriting imitation with latent diffusion models. *International Journal of Computer Vision (IJCV)*, 133(10):7054–7075.
- Muth, M., Peer, M., Kleber, F., and Sablatnig, R. (2024). Maximizing data efficiency of handwriting recognition models by synthetic text. In *Document Analysis Systems (DAS)*, pp. 295–311.
- Nikolaidou, K., Retsinas, G., Christlein, V., Seuret, M., Sfikas, G., Smith, E. B., Mokayed, H., and Liwicki, M. (2023). WordStylist: Styled verbatim handwritten text generation with latent diffusion models. In *International Conference on Document Analysis and Recognition (ICDAR)*, pp. 384–401.
- Nikolaidou, K., Retsinas, G., Sfikas, G., and Liwicki, M. (2024). DiffusionPen: Towards controlling the style of handwritten text generation. In *European Conference on Computer Vision (ECCV)*, pp. 417–434.
- Peer, M., Sablatnig, R., and Kleber, F. (2025). Towards the influence of text quantity on writer retrieval. In *International Conference on Document Analysis and Recognition (ICDAR)*, pp. 129–145.
- Peer, M., Kleber, F., and Sablatnig, R. (2024). SAGHOG: Self-supervised autoencoder for generating HOG features for writer retrieval. In *International Conference on Document Analysis and Recognition (ICDAR)*, pp. 121–138.
- Peer, M., Kleber, F., and Sablatnig, R. (2026). BullingerDB: A Dataset for Handwritten Text Recognition and Writer Retrieval. Accepted for presentation at the *International Conference on Document Analysis and Recognition (ICDAR)*. arXiv:2605.30235.
- Pippi, V., Cascianelli, S., and Cucchiara, R. (2023). Handwritten text generation from visual archetypes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22458–22467.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10684–10695.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520.

- Sauvola, J. and Pietikäinen, M. (2000). Adaptive document image binarization. *Pattern Recognition*, 33(2):225–236.
- Song, J., Meng, C., and Ermon, S. (2021). Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*.